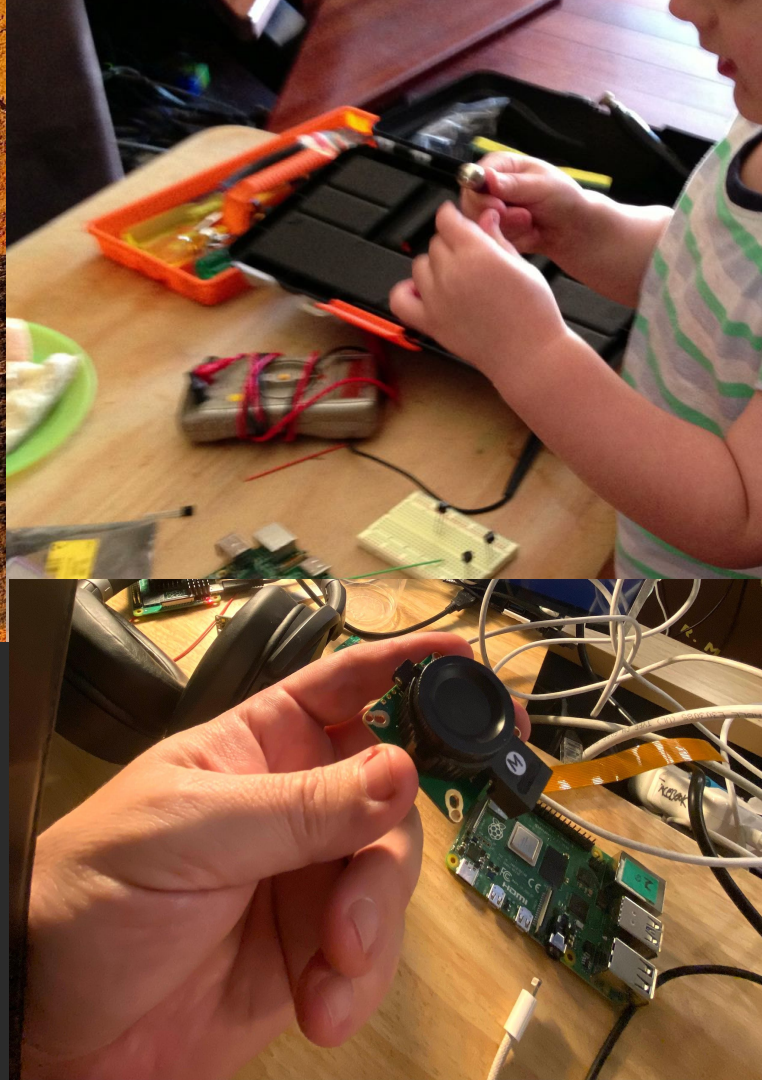
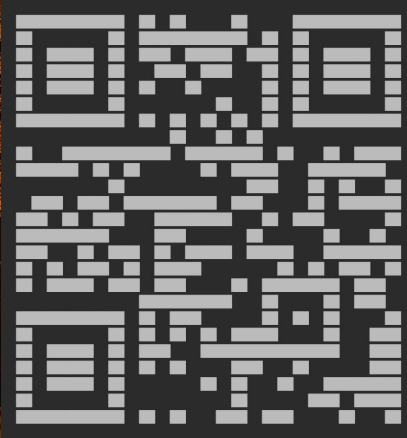


Raspberry Pi Live Streaming

Kevin Staunton-Lambert
@kevleyski

pyrmontbrewery.com.au

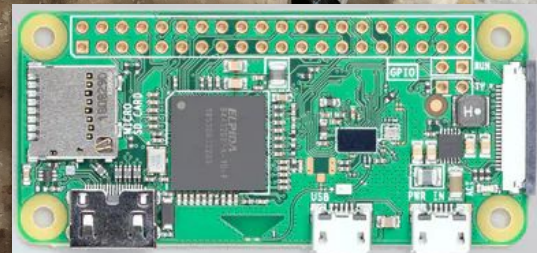
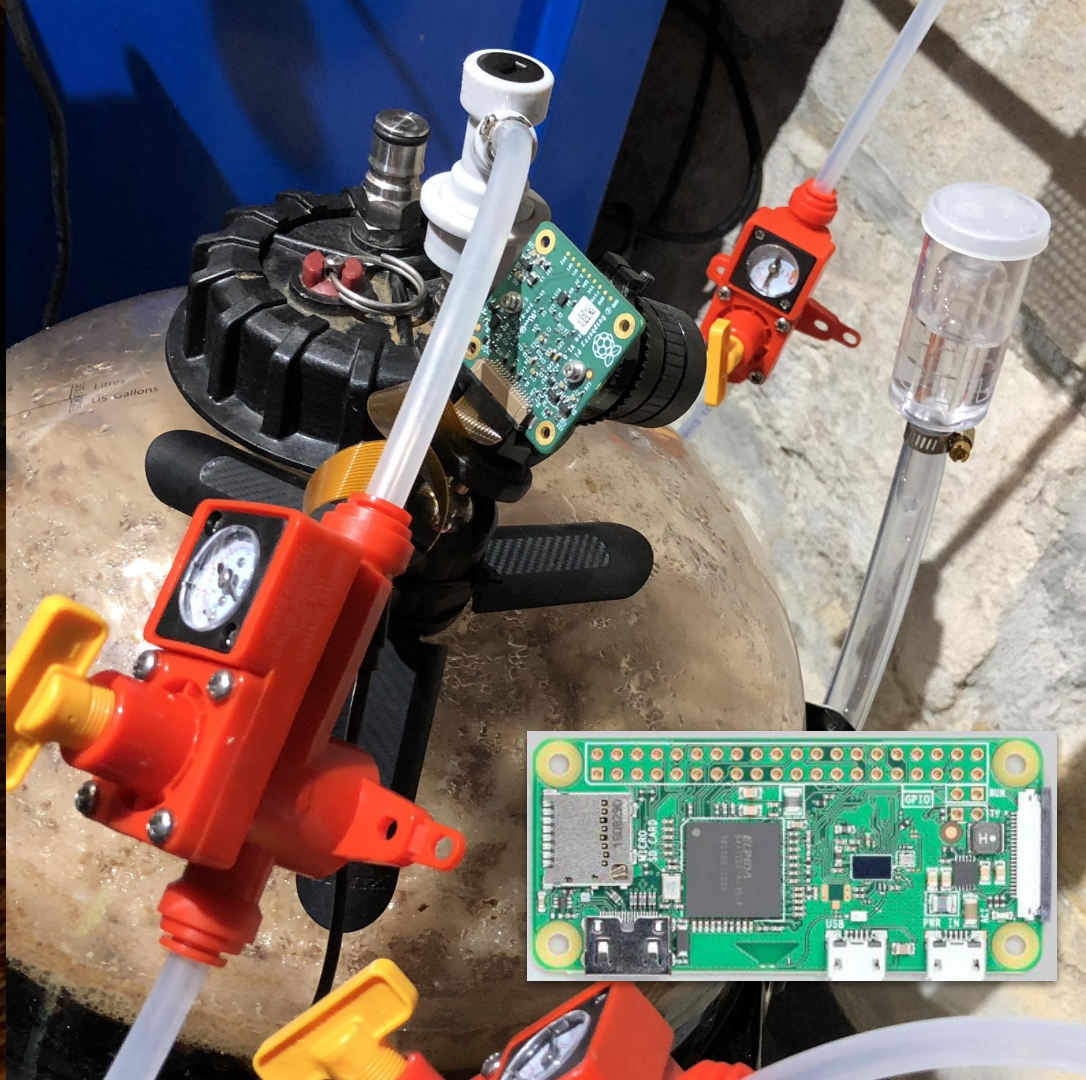


BeerCAM!™

Low latency beer
brewing bubble counter!

Raspberry Pi MIPI/CSI2
HD Camera + \$5 PiZero

FFmpeg w/ SRT



+ \$35 for a Raspberry Pi 4 for compute (2GB)



A photograph of a cardboard box. On the left side, there is a Raspberry Pi logo sticker (a red raspberry with green leaves) and a small white label with black text and a barcode. The label text includes "Raspberry Pi", "Model B", "Serial No.", "Date of Birth", and "© 2007 Pirbright Science". A red circle is drawn on the right side of the box, highlighting a dark, irregular shape that appears to be a hole or a piece of tape.

Add OpenCV to detect motion



OpenCV... first we blur source to reduce noise



OpenCV... (#2 Canny edge detect + contour count)

contours=486



OpenCV... crunch mean histogram variances (C++)

```
@700ms:avg_hist=77.6122:histdelta=77.6122:avg_edges=4  
80:frame_edges=480
```

```
@2100ms:avg_hist=77.613:histdelta=77.6139:avg_edges=4  
81:frame_edges=483
```

```
@4900ms:avg_hist=77.5504:histdelta=77.4251:avg_edges=  
483:frame_edges=486
```

Mean histogram=77.5

Average number of edges=482

Add MQTT for remote control (mosquito)

Change exposure/contrast/brightness

PTZ cams can change pan/tilt/focus/iris aperture

Add sunwait for daylight based cron jobs

```
set_cam () {  
    v4l2-ctl -c brightness=10 --device $1  
}
```

```
# Wait for Sydney sunrise  
sunwait wait rise 33.869561S 151.194614E  
set_cam /dev/video2
```

Super *fast* Rust code to generate LL-HLS deltas

```
#EXTM3U
#EXT-X-VERSION:9
#EXT-X-TARGETDURATION:2
#EXT-X-INDEPENDENT-SEGMENTS
#EXT-X-PART-INF:PART-TARGET=0.5
#EXT-X-MEDIA-SEQUENCE:816795955
#EXT-X-PROGRAM-DATE-TIME:2021-10-07T07:31:50.000Z
#EXT-X-SKIP:SKIPPED-SEGMENTS=22
#EXTINF:2,
/seg/dca63211b9a6/1633591966_20211007_073246.ts
#EXT-X-PART:DURATION=0.5, INDEPENDENT=YES, URI="/seg/dca63211b9a6/16
33591968_20211007_073248.ts?part", BYTERANGE="49632@0"
#EXT-X-PART:DURATION=0.5, URI="/seg/dca63211b9a6/1633591968_2021100
7_073248.ts?part", BYTERANGE="49632@49632"
...
```


Super *fast* Rust code to generate LL-HLS deltas

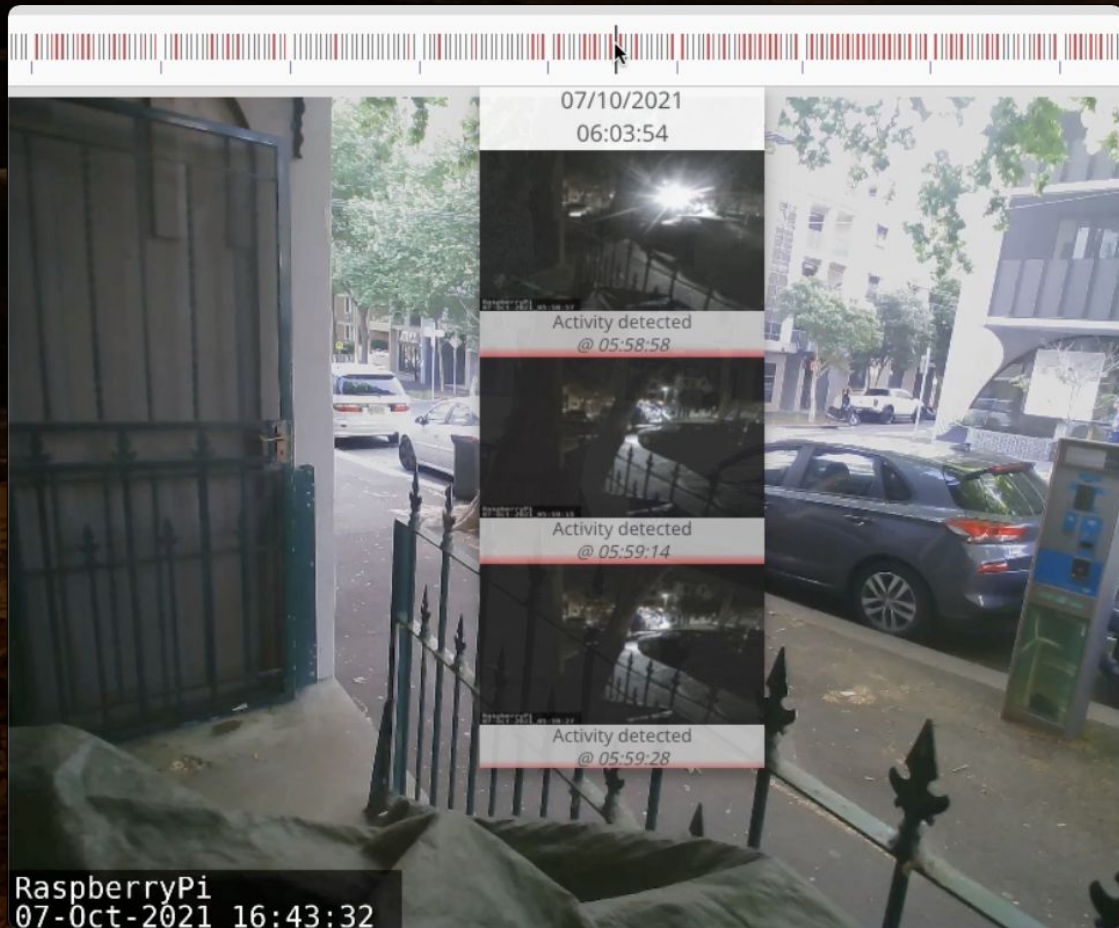
...

```
#EXT-X-PART:DURATION=0.5,INDEPENDENT=YES,URI="/seg/dca63211b9a6/1633591968_20211007_073248.ts?part",BYTERANGE="49632@99264"
#EXT-X-PART:DURATION=0.5,URI="/seg/dca63211b9a6/1633591968_20211007_073248.ts?part",BYTERANGE="49632@148896"
#EXT-X-PROGRAM-DATE-TIME:2021-10-07T07:32:48.000Z
#EXTINF:2,
/seg/dca63211b9a6/1633591968_20211007_073248.ts
#EXT-X-SERVER-CONTROL:CAN-BLOCK-RELOAD=YES,CAN-SKIP-UNTIL=18,PART-HOLD-BACK=1.5
#EXT-X-PRELOAD-HINT:TYPE=PART,URI="/seg/dca63211b9a6/1633591970_20211007_073250.ts"
```

hls.js low latency
player
+ thumbnail
Scrubber

Jump between
activity markers

Weeks of
footage (cloud or
local storage is optional)



Remote viewer latency: 2.36sec

(end player)

@51.88 seconds
Chrome + hls.js
LL-HLS

(source)

@53.92
seconds

RasPi4
With
MIPI/
CSI2
Camera
Pointing
at iMac

@54.24 seconds
(iMac) Source
material wall clock
pyrmontbrewery.com/time.html

@52.44 seconds
SRT @ StreamShark Melbourne
-> Google Cloud running in-house
Rust w/ FFmpeg code

@53.92 seconds
Raspberry Pi
HDMI output

and yes I have a hammock

Remote: **Syd** > Mel > Google US > Syd = **2.36sec**

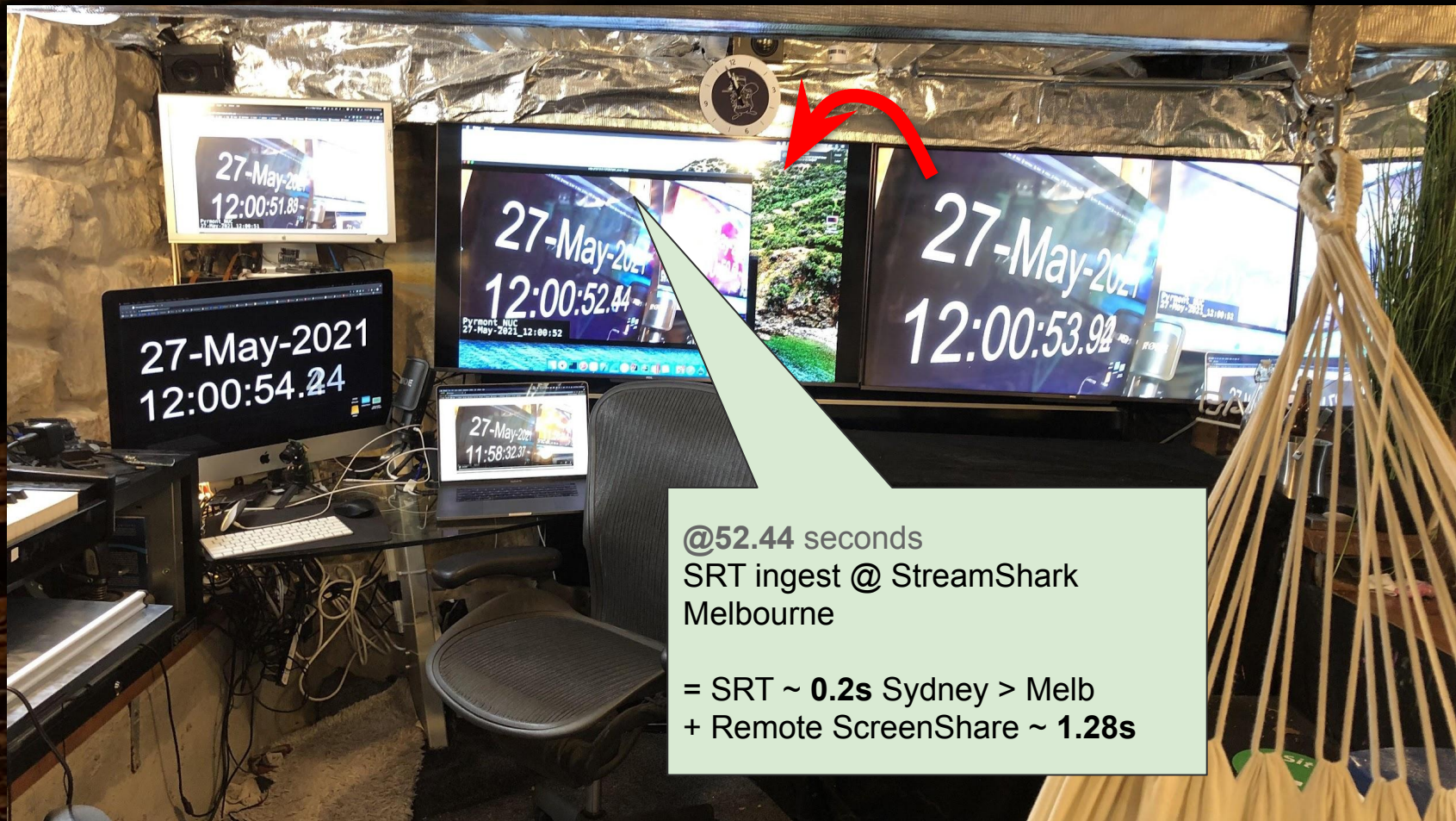


RasPi4
with
MIPI/
CSI2
Camera
Pointed
at iMac

@54.24 seconds
(iMac) Source
material wall clock
[pyrmontbrewery.com
/time.html](http://pyrmontbrewery.com/time.html)

@53.92 seconds
Raspberry Pi
HDMI output
= Cam latency
0.32s

Remote: Syd > **Mel** > Google US > Syd = **2.36sec**



@52.44 seconds
SRT ingest @ StreamShark
Melbourne

= SRT ~ **0.2s** Sydney > Melb
+ Remote ScreenShare ~ **1.28s**

Remote: Syd > Mel > **Google US** > Syd = **2.36sec**

(end player)

@**51.88** seconds

Chrome + hls.js

LL-HLS

= Cloud Compute

0.56s



DIY, get going with a Raspberry Pi (headless setup)

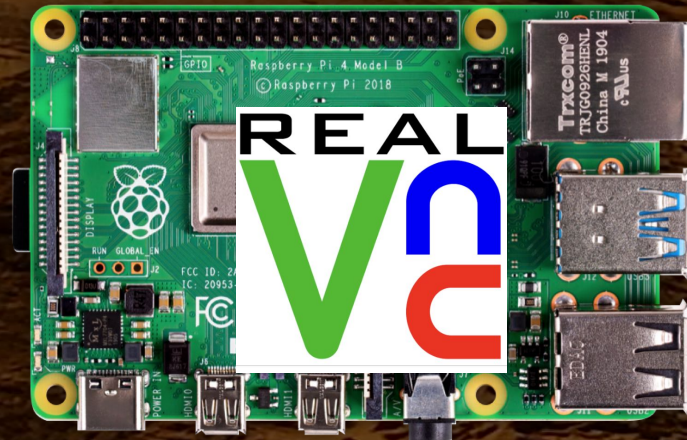
Copy the Raspberry Pi OS Linux distribution .iso to SD card

```
$ sudo dd if=/dev/rdisk1 bs=1m | wget https://downloads.raspberrypi.org/raspbian_full_latest
```

You don't need a keyboard, mouse or a monitor ;-)

```
$ touch /Volumes/boot/ssh
$ vi /Volumes/boot/wpa_supplicant.conf
country=AU
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
network={
    ssid="YOUR_WIFI_NAME_BSSID"
    psk="YOUR_WIFI_PASSWORD"
}
$ sudo apt install realvnc-vnc-server realvnc-vnc-viewer
```

Docker / remote SSH runs well too



libcamera (<https://github.com/kevleyski/libcamera>)

Open Source replacement for Broadcom
proprietary raspistill/raspivid

libcamera-still

Auto shutter thumbnails, stills can be passed to OpenCV

libcamera-vid / gstreamer libcamerasrc

Can stream via gstreamer to other processes that ingest the video
and optionally analyse it

libcamera

<code>--list-cameras [=arg(=1)] (=0)</code>	Lists the available cameras attached to the system.
<code>--camera arg (=0)</code>	Chooses the camera to use.
<code>-c [--config] [=arg(=config.txt)]</code>	Read the options from a file.
<code>--width arg (=0)</code>	Set the output image width
<code>--height arg (=0)</code>	Set the output image height
<code>-t [--timeout] arg (=5000)</code>	Time (in ms) for which program runs
<code>-o [--output] arg</code>	Set the output file name
<code>--post-process-file arg</code> post-processing	Set the file name for configuring the
<code>--rawfull [=arg(=1)] (=0)</code>	Force use of full resolution raw frames

libcamera

<code>-n [--nopreview] [=arg(=1)] (=0)</code>	Do not show a preview window
<code>-p [--preview] arg (=0,0,0,0)</code> <code>x,y,width,height e.g. 0,0,640,480</code>	Set the preview window dimensions, given as
<code>-f [--fullscreen] [=arg(=1)] (=0)</code>	Use a fullscreen preview window
<code>--qt-preview [=arg(=1)] (=0)</code> CPU load, fullscreen not supported)	Use Qt-based preview window (WARNING: causes heavy
<code>--hflip [=arg(=1)] (=0)</code>	Request a horizontal flip transform
<code>--vflip [=arg(=1)] (=0)</code>	Request a vertical flip transform
<code>--rotation arg (=0)</code>	Request an image rotation, 0 or 180
<code>--roi arg (=0,0,0,0)</code>	Set region of interest e.g. 0.25,0.25,0.5,0.5

libcamera

<code>--shutter arg (=0)</code>	Set a fixed shutter speed in microseconds
<code>--analoggain arg (=0)</code>	Set a fixed gain value (synonym for 'gain' option)
<code>--gain arg</code>	Set a fixed gain value
<code>--metering arg (=centre)</code>	Metering mode (centre, spot, average, custom)
<code>--exposure arg (=normal)</code>	Set the exposure mode (normal, sport)
<code>--ev arg (=0)</code>	Set the EV exposure compensation, 0 = no change
<code>--awb arg (=auto)</code>	AWB mode (auto, incandescent, tungsten, fluorescent, indoor, daylight, cloudy, custom)
<code>--awbgains arg (=0,0)</code>	Set explicit red and blue gains

libcamera

`--flush [=arg(=1)] (=0)`

Flush output data as soon as possible

`--wrap arg (=0)`

Reset the counter when it reaches this

`--brightness arg (=0)`

Brightness of the output images, -1.0 to 1.0

`--contrast arg (=1)`

Contrast of the output image, 1.0 = normal contrast

`--saturation arg (=1)`

Colour saturation, 1.0 = normal and 0.0 = greyscale

`--sharpness arg (=1)`

Sharpness of the output image, where 1.0 = normal

`--framerate arg (=30)`

Set the fixed framerate for preview and video modes

`--denoise arg (=auto)`

Denoise mode: auto, off, cdn_off, cdn_fast, cdn_hq

libcamera

<code>--viewfinder-width arg (=0)</code>	Width of viewfinder frames from the camera
<code>--viewfinder-height arg (=0)</code>	Height of viewfinder frames from the camera
<code>--tuning-file arg (=)</code>	Name of camera tuning file to use
<code>--lores-width arg (=0)</code>	Width of low resolution frames
<code>--lores-height arg (=0)</code>	Height of low resolution frames
<code>--mode arg</code>	Mode W:H:bit-depth:packing, P (packed) or U
<code>--viewfinder-mode arg</code>	Preview as W:H:bit-depth:packing, packing is P or U
<code>-q [--quality] arg (=93)</code>	Set the JPEG quality parameter
<code>-x [--exif] arg</code>	Add these extra EXIF tags to the output file

libcamera

`-e [ --encoding ] arg (=jpg)`

Encoding, either jpg, png, rgb, bmp or yuv420

`-r [ --raw ] [=arg(=1)] (=0)`

Also save raw file in DNG format

`--latest arg`

Create a symbolic link to latest with this name

`--immediate [=arg(=1)] (=0)`

Perform first capture immediately, no preview phase

`-k [ --keypress ] [=arg(=1)] (=0)`

Perform capture when ENTER pressed

`-s [ --signal ] [=arg(=1)] (=0)`

Perform capture when signal received

libcamera-still specific

<code>--timelapse arg (=0)</code>	Time interval (in ms) between timelapse captures
<code>--framestart arg (=0)</code>	Initial frame counter value for timelapse captures
<code>--datetime [=arg(=1)] (=0)</code>	Use date format for output file names
<code>--timestamp [=arg(=1)] (=0)</code>	Use system timestamps for output file names
<code>--restart arg (=0)</code>	Set JPEG restart interval
<code>--thumb arg (=320:240:70)</code>	Thumbnail parameters width:height:quality, or none

libcamera-vid specific

<code>-b [--bitrate] arg (=0)</code>	Bitrate for encoding, bits/second (h264 only)
<code>--profile arg</code>	Set the encoding profile (h264 only)
<code>--level arg</code>	Set the encoding level (h264 only)
<code>-g [--intra] arg (=0)</code>	Set the intra frame period (h264 only)
<code>--inline [=arg(=1)] (=0)</code>	Force PPS/SPS header with every I frame (h264 only)
<code>--codec arg (=h264)</code>	Set the codec to use, either h264, mjpeg or yuv420
<code>--save-pts arg</code>	Save a timestamp file with this name
<code>-q [--quality] arg (=50)</code>	Set the MJPEG quality parameter (mjpeg only)
<code>-l [--listen] [=arg(=1)] (=0)</code>	Listen for an incoming client network connection before sending data to the client

libcamera-vid specific

`-i [ --initial ] arg (=record)`

Use 'pause' to pause the recording at startup

`--split [=arg(=1)] (=0)`

Create a new output file every time recording is paused and then resumed

`--segment arg (=0)`

Break the recording into files of approximately this many milliseconds

`--circular [=arg(=4)] (=0)`

Write output to a circular buffer of the given size (in MB) which is saved on exit

`--frames arg (=0)`
will override any timeout set.

Run for the exact number of frames specified. This

Thanks! [GitHub/PymontBrewery](https://github.com/PymontBrewery)

DIY Webcams

Short history of webcams and how to roll your own

<https://tinyurl.com/2jjt644x>

Why-Pi?

Why and how the Raspberry Pi came to be

<https://tinyurl.com/2cf4k4br>

DIY Webcams

Short history of webcams and how to roll your own

Kev Staunton-Lambert



Why-Pi

Why and how the Raspberry Pi came to be

Prepared for Jaycar Maker Hub (Sydney)
Raspberry Jam, 3.14 14th March 2020

Kev Staunton-Lambert

pymontbrewery.com

